# Design Patterns in C++

Named, proven solutions to recurring design problems. The five most common (Singleton, Factory, Strategy, Observer, State), how modern C++ changes them, and SOLID.

## What patterns are and are not

A pattern is a **vocabulary**: "this is an Observer" conveys structure and intent. Three families:

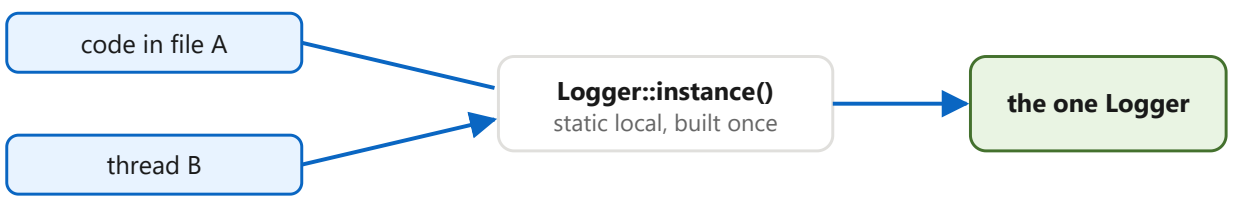
| FAMILY | CONCERNED WITH | EXAMPLES |
|---|---|---|
| **Creational** | how objects are created | Singleton, Factory, Builder |
| **Structural** | how objects are composed | Adapter, Decorator, Proxy, PIMPL |
| **Behavioural** | how objects interact and share responsibility | Strategy, Observer, State, Command, Visitor |

NOT GOALS Each pattern adds indirection: use one only for a real problem, and know *when not to use it*.

MODERN C++ SHORTENS MANY A lambda or `std::function` replaces Strategy and Command hierarchies; `std::visit` replaces Visitor; smart pointers are a Proxy; RAII is built in.

---

## Singleton




Thread-safe creation since C++11; its methods still need their own locking.

**Intent:** exactly one instance with global access (logger, config).

```
class Logger {
public:
    static Logger& instance() {
        static Logger inst;              // created the first time control
passes here
        return inst;                     //  ... and the same object is
returned forever after
    }
    Logger(const Logger&) = delete;      // forbid copies: there must be only
one
    Logger& operator=(const Logger&) = delete;

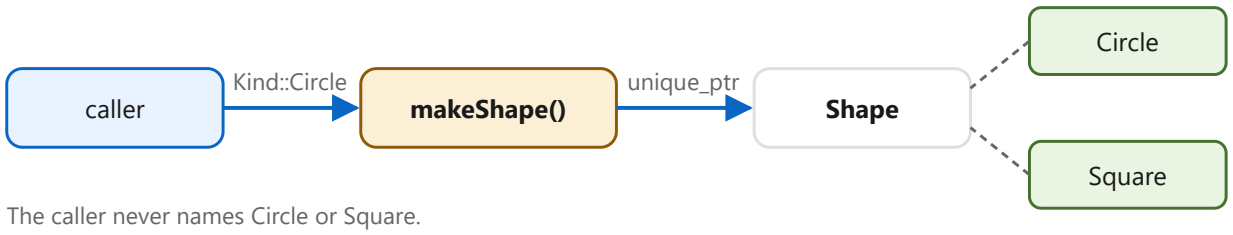
    void log(const std::string& msg) { std::cout << msg << '\n'; }
private:
    Logger() { std::cout << "Logger constructed\n"; }   // private: nobody
else can create one
};

Logger::instance().log("hello");
```

The **Meyers Singleton**:

- **Thread safe creation:** a function-local `static` is initialized exactly once since C++11 ("magic statics"): no mutex or double-checked locking. Calls on the object still need their own locking.
- **Lazy:** built on first use, avoiding static initialization order problems.
- **Drawbacks (say them):** global state in disguise, hidden dependencies, hard to test with a fake, unspecified destruction order at exit.
- **Alternative:** create one object in `main` and pass it by reference (dependency injection).

## Factory




The caller never names Circle or Square.

New type means editing the factory, unless it keeps a registry.

**Intent:** create objects without the caller naming the concrete class; it gets something usable only through an interface.

```
struct Shape {
    virtual double area() const = 0;
    virtual ~Shape() = default;
};
struct Circle : Shape { double r; explicit Circle(double r) : r(r) {} double
area() const override { return 3.14159 * r * r; } };
struct Square : Shape { double s; explicit Square(double s) : s(s) {} double
area() const override { return s * s; } };

enum class Kind { Circle, Square };

std::unique_ptr<Shape> makeShape(Kind k, double size) {       // the factory
function
    switch (k) {
        case Kind::Circle: return std::make_unique<Circle>(size);
        case Kind::Square: return std::make_unique<Square>(size);
    }
    return nullptr;
}

auto s = makeShape(Kind::Circle, 2.0);       // caller sees only Shape
std::cout << s->area();
```

- **Return** `unique_ptr`: clear ownership, no leaks, convertible to `shared_ptr`.
- The base needs a **virtual destructor**.
- A new type means editing the factory. Avoid with a *registry*: `map<string, function<unique_ptr<Shape>(double)>>` (Open/Closed).

- **Factory Method**: subclasses decide what to create. **Abstract Factory**: families of related objects (a matching `Button` and `Checkbox` per platform).

---

## Strategy




The client picks the algorithm; modern C++ often uses a lambda instead of a class.

**Intent:** interchangeable algorithms chosen *at runtime* without changing the user (sort order, pricing, retry policy).

```
// 1. Classic: an interface and several implementations
struct PricingStrategy {
    virtual double apply(double price) const = 0;
    virtual ~PricingStrategy() = default;
};
struct Regular   : PricingStrategy { double apply(double p) const override {
return p; } };
struct Member    : PricingStrategy { double apply(double p) const override {
return p * 0.90; } };

class Checkout {
    std::unique_ptr<PricingStrategy> strategy_;
public:
    explicit Checkout(std::unique_ptr<PricingStrategy> s) :
strategy_(std::move(s)) {}
    double total(double price) const { return strategy_->apply(price); }
};

// 2. Modern: the strategy IS a callable. No class hierarchy needed.
class Checkout2 {
    std::function<double(double)> pricing_;
public:
    explicit Checkout2(std::function<double(double)> f) :
pricing_(std::move(f)) {}
    double total(double price) const { return pricing_(price); }
};
```

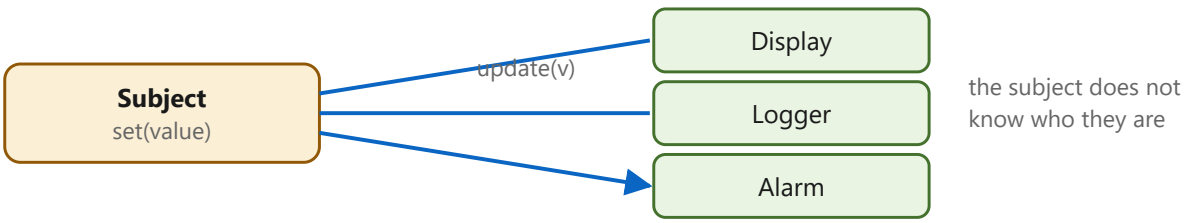
```
Checkout2 c([](double p) { return p * 0.70; });     // a lambda is a strategy

// 3. Compile-time: the strategy is a template parameter (policy-based
design): zero overhead
template <typename Pricing> class Checkout3 { /* ... Pricing{}(price) ... */
};
```

> **STRATEGY VS STATE** Similar shape, different intent. **Strategy**: the *client* picks the algorithm; strategies are independent. **State**: the object changes its own behaviour as its state changes, and states trigger the transitions.

---

## Observer




Watch for dangling observers, and iterate over a copy while notifying.

**Intent:** a *subject* notifies its *observers* on change without knowing who they are (events, signals, pub/sub).

```
class Observer {
public:
    virtual void update(double value) = 0;
    virtual ~Observer() = default;
};

class Subject {
    std::vector<Observer*> observers_;
public:
    void attach(Observer* o) { observers_.push_back(o); }
    void detach(Observer* o) {
observers_.erase(std::remove(observers_.begin(), observers_.end(), o),
observers_.end()); }
    void set(double v) {
        value_ = v;
        auto copy = observers_;                          // iterate a COPY: an
```

```
    // observer may detach itself in update()
        for (auto* o : copy) o->update(v);
    }
private:
    double value_ = 0;
};

// Modern, lighter version: observers are just callbacks
std::vector<std::function<void(double)>> listeners;
listeners.push_back([](double t) { std::cout << "temp = " << t << '\n'; });
```

CLASSIC OBSERVER BUGS

**Dangling observer:** subject calls a destroyed object. Detach in the destructor (RAII) or store `weak_ptr`.

**Modifying the list while notifying:** iterate a copy.

**Threads:** lock, but never while calling observers (deadlock risk).

---

## State

**Intent:** behaviour changes with internal state, replacing scattered `if (state == ...)` checks (a turnstile).

```
// 1. Simplest: an enum and a switch. Fine for a few states.
enum class State { Locked, Unlocked };
class Turnstile {
    State s_ = State::Locked;
public:
    void coin() { s_ = State::Unlocked; }
    void push() { if (s_ == State::Unlocked) s_ = State::Locked; else
alarm(); }
};

// 2. Classic State pattern: one class per state
struct TurnstileState {
    virtual std::unique_ptr<TurnstileState> coin() = 0;     // each state
returns the NEXT state
    virtual std::unique_ptr<TurnstileState> push() = 0;
    virtual ~TurnstileState() = default;
};
```

```
// 3. Modern: std::variant of state types + std::visit; the compiler checks
every state is handled.
using StateV = std::variant<Locked, Unlocked>;
```

Use it when valid operations depend on "what mode am I in": connections, parsers, protocol handlers. Explicit transitions make illegal states unrepresentable and testable.

---

## SOLID principles

**SOLID** are five design principles. Most patterns in this lesson exist to serve one of them, so use this table to connect each pattern back to the principle it serves.

| | PRINCIPLE | IN ONE SENTENCE | PATTERN THAT SERVES IT |
|---|---|---|---|
| **S** | Single Responsibility | a class has one reason to change | split parsing, logic and I/O into separate classes |
| **O** | Open/Closed | open for extension, closed for modification: add behaviour by adding code, not editing old code | Strategy, Factory registry, Decorator |
| **L** | Liskov Substitution | a derived object must be usable wherever the base is expected without breaking correctness | correct inheritance hierarchies |
| **I** | Interface Segregation | prefer several small interfaces over one fat one | narrow abstract classes |
| **D** | Dependency Inversion | depend on abstractions, not concrete classes; inject dependencies | Strategy, Observer, dependency injection (instead of Singleton) |

## Other patterns worth recognising

| PATTERN | IDEA | WHERE YOU ALREADY SEE IT IN C++ |
|---|---|---|
| **RAII** | object owns a resource and releases it in its destructor | `vector`, `unique_ptr`, `lock_guard` |
| **Adapter** | wrap an interface so it matches the one clients expect | `std::stack` (adapts `deque`), wrapping a C library |
| **Decorator** | wrap an object to add behaviour while keeping its interface | buffered/compressed stream wrappers, logging wrappers |
| **Proxy** | a stand-in that controls access to another object | `shared_ptr`, `vector<bool>::reference` |
| **Command** | turn a request into an object (queue it, undo it) | task queues of `std::function`, thread pools |
| **Iterator** | traverse a collection without exposing its structure | the entire STL |
| **Visitor** | add operations to a type hierarchy without changing it | `std::visit` with `std::variant` |
| **Builder** | construct a complex object step by step | fluent config objects: `Req().url(u).timeout(5).build()` |
| **PIMPL** | hide implementation behind a pointer to keep headers stable and compile times low | library public headers |
| **CRTP** | a class derives from a template instantiated with itself: static polymorphism | mix-ins, `std::enable_shared_from_this` |

```cpp
// CRTP: compile-time polymorphism, no vtable
template <typename Derived>
struct Shape {
    double area() const { return static_cast<const Derived*>(this)-
>areaImpl(); }
};
struct Circle : Shape<Circle> {
    double r = 1;
    double areaImpl() const { return 3.14159 * r * r; }
};
```

WITH REAL CODE Command, Decorator, Adapter, Builder, Visitor with `std::variant` and a session state machine are in More Patterns.

## Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

| SECTION | BOOK | PART OF THE BOOK | PDF PAGES |
|---|---|---|---|
| **Observer** | Effective Modern C++ | Item 20, Use std::weak_ptr for shared_ptr-like pointers that can dangle | 152 to 157 |
| **Other patterns worth recognising** | Effective Modern C++ | Item 22, Pimpl: define special member functions in the implementation file | 165 to 174 |